Embedded Systems Study Mastery Checklist

BTech CSE → CDAC Embedded → Industry → Masters (Germany)

Target roles: Embedded Software Engineer · Linux Device Driver Engineer · BSP / Platform Engineer · Firmware Engineer

How to use this guide:

• Print and pin to your wall. Tick off each item only when you can explain it out loud or write code for it — not just when you've read it.
• MUST MASTER and MUST KNOW WELL are your primary goals. Aim to complete both before interviews.
• Items in bold are highest-priority — if short on time, never skip these.
• Notes in italics give context — read them once, don't memorise them.

Tier	Expectation	Interview behavior
MUST MASTER	Write code from memory, debug confidently	Grilled hard — failure here = rejection
MUST KNOW WELL	Explain clearly, write pseudocode	Will definitely come up — blanks hurt
BE DECENT	Explain concept, know the tradeoffs	Depth expected at senior level
AWARENESS	Know it exists, why it matters	Asked at top MNCs for signals of range

MUST MASTER

C Programming — The Language of Embedded

Every interview starts here. You must write correct C on a blank editor — no IDE, no autocomplete.

1.1 Pointers & Memory

- [ ] **Pointer arithmetic — incrementing int* vs char* vs struct*, what address changes by**
- [ ] Declare and use: int *p, *p++, p+2, p[n] — all mean the same thing, know why
- [ ] **Pointer to pointer (**p) — declare, dereference, modify through two levels**
- [ ] **Array of pointers vs pointer to array — declaration syntax, how indexing differs**
- [ ] **Function pointers — declare, typedef, store in struct, use as callback**
- [ ] Example: void (*handler)(int) = isr_handler; handler(5);
- [ ] **All 4 const+pointer combos — int *p, const int *p, int * const p, const int * const p**
- [ ] void* — cast to/from, alignment rules, why you can't dereference directly
- [ ] Dangling pointer — what causes it, how to detect, set to NULL after free
- [ ] Wild pointer vs null pointer — difference, how each causes faults
- [ ] Memory leak — identify in a code snippet, how valgrind/sanitizers catch it

1.2 Embedded Keywords (volatile, const, static, register, extern)

- [ ] **volatile — why MCU peripheral registers need it; why ISR-shared variables need it**
- [ ] Rule: any variable written by hardware OR another execution context needs volatile
- [ ] **const — placing variables in Flash/ROM, read-only parameters, const correctness**
- [ ] **static (file scope) — limits symbol visibility to one translation unit**
- [ ] **static (function scope) — persistent local variable, initialized once, survives calls**
- [ ] extern — declare without define; link across files; understand one-definition rule
- [ ] inline — hint to compiler to expand in-place; not the same as macro, has type safety
- [ ] register — hint (mostly ignored today); know what it historically meant
- [ ] restrict — tells compiler no aliasing; allows optimization on pointer parameters

1.3 Bit Manipulation

- [ ] **Set bit N: reg |= (1U << N)**
- [ ] **Clear bit N: reg &= ~(1U << N)**
- [ ] **Toggle bit N: reg ^= (1U << N)**
- [ ] **Check bit N: (reg >> N) & 1U**
- [ ] **Bitmask to configure a register field — extract, clear, set pattern**
- [ ] **Endianness — big-endian vs little-endian, how to detect at runtime, byte swap macro**
- [ ] Bit fields in structs — packing behavior, alignment, portability caveats
- [ ] Count set bits (Kernighan method: n &= n-1 in a loop)
- [ ] Check if power of 2: n && !(n & (n-1))
- [ ] Reverse bits of a 32-bit integer
- [ ] Find rightmost set bit: n & (-n)

1.4 Memory Layout (Text / Data / BSS / Stack / Heap)

- [ ] **.text — compiled machine code lives here (ROM/Flash)**
- [ ] **.data — initialized global/static variables (copied from Flash to RAM at startup)**
- [ ] **.BSS — zero-initialized globals/statics (zeroed by startup code, no Flash storage)**
- [ ] **Stack — grows downward; local variables, return addresses, function call frames**
- [ ] **Heap — grows upward; dynamic allocation (malloc/free)**
- [ ] **Startup code flow: copy .data from Flash → zero .BSS → call main()**
- [ ] **Stack overflow — what causes it, how RTOS stack watermarks detect it**
- [ ] malloc/calloc/realloc/free — behavior, difference between calloc and malloc
- [ ] Memory fragmentation — why it matters in long-running embedded systems
- [ ] **Why dynamic allocation is avoided in safety-critical embedded code**
- [ ] Linker script — MEMORY and SECTIONS blocks, what OUTPUT_SECTION does

1.5 Data Structures in C (No STL — write from scratch)

- [ ] **Circular buffer (ring buffer) — implement with head, tail, count; handle full and empty**
- [ ] Must be able to write this cold — extremely common in embedded interviews
- [ ] **Linked list — singly and doubly; insert at head/tail, delete node, reverse in-place**
- [ ] **Finite state machine — implement with enum for states + function pointer dispatch table**
- [ ] Know both switch-case FSM and function-pointer-table FSM
- [ ] Stack using array — push, pop, overflow check
- [ ] Queue using circular array — enqueue, dequeue
- [ ] **Struct alignment — sizeof() surprises, __attribute__((packed)), padding rules**

1.6 Preprocessor, Macros & Compiler Behavior

- [ ] **Macros vs inline functions — macros have no type safety, no scope; inlines do**
- [ ] Multi-statement macro safety: do { ... } while(0) pattern
- [ ] Stringification (#) and token pasting (##) in macros
- [ ] #pragma once vs include guards — how each prevents double inclusion
- [ ] **Integer overflow — signed overflow is UB in C; unsigned wraps (defined)**
- [ ] **Signed vs unsigned comparison pitfalls — -1 > 0u is true!**
- [ ] Sequence points and undefined behavior — i++ + i++ is UB
- [ ] GCC flags: -O0/-O2/-Os (size optimize), -Wall -Wextra, -fno-strict-aliasing
- [ ] Compilation pipeline: .c → preprocessor → .i → compiler → .s → assembler → .o → linker → ELF

MUST MASTER

Basic Electronics — The Physical Foundation

You are writing code that controls physical hardware. Understanding the electronics beneath you is non-negotiable for bring-up and debugging roles.

2.1 Fundamental Laws & Concepts

- [ ] **Ohm's Law: V = IR — apply to calculate voltage drop, current draw, resistor values**
- [ ] **Kirchhoff's Current Law (KCL) — sum of currents into a node = 0**
- [ ] **Kirchhoff's Voltage Law (KVL) — sum of voltages around a loop = 0**
- [ ] **Power: P = VI = I^2R = V^2/R — calculate power dissipation in a resistor**
- [ ] **Voltage divider: Vout = Vin × R2/(R1+R2) — used for ADC input scaling**
- [ ] Current divider — parallel resistor current splitting
- [ ] **Series vs parallel resistors — equivalent resistance formulas**
- [ ] Series vs parallel capacitors — equivalent capacitance formulas

2.2 Components You Will Encounter Daily

- [ ] **Resistor — purpose (limit current, pull-up/down), read color code, tolerance**
- [ ] **Pull-up resistor — why I2C lines need them; calculate value from Vcc, bus capacitance, speed**
- [ ] **Pull-down resistor — ensure GPIO reads LOW when floating**
- [ ] **Capacitor — bypass/decoupling cap purpose (absorb transients near IC VCC pin)**
- [ ] Rule of thumb: 100nF ceramic cap on every IC power pin, bulk cap per board section
- [ ] Capacitor — filtering (RC low-pass) and timing (RC time constant $\tau = RC$)
- [ ] Inductor — oppose change in current; used in switching regulators and EMC filters
- [ ] **Diode — one-way current flow; V_f drop (~0.7V silicon, ~0.3V Schottky)**
- [ ] Zener diode — voltage regulation; TVS diode for ESD protection
- [ ] **LED — current-limiting resistor calculation: R = (Vcc - V_f) / I_f**
- [ ] Transistor (BJT) — NPN/PNP; saturation vs active region; use as switch (not amplifier)
- [ ] **MOSFET — N-channel/P-channel; gate voltage controls drain-source; used for high-side/low-side switching**
- [ ] Crystal oscillator — provides MCU clock; load capacitors, startup time

2.3 Digital Logic & Signal Levels

- [ ] **Logic families: 3.3V vs 5V — NEVER connect 5V GPIO directly to 3.3V MCU input**
- [ ] **Logic level shifting — voltage divider method vs level shifter IC**
- [ ] CMOS vs TTL input/output thresholds — Vil, Vih, Vol, Voh definitions
- [ ] **Push-pull output vs open-drain output — when to use each**
- [ ] Open-drain: can only pull low; needs external pull-up; used for I2C, wired-OR configs
- [ ] **Tri-state / high-impedance — what it means, when a GPIO is floating**
- [ ] Schmitt trigger input — hysteresis; why it prevents noise-induced toggling on slow signals
- [ ] Fan-out — how many inputs one output can drive; exceeded fan-out causes voltage droop

2.4 Analog Concepts Relevant to Embedded

- [ ] **ADC — resolution (bits → LSB voltage), sampling rate, Nyquist theorem**
- [ ] 12-bit ADC on 3.3V: LSB = 3.3/4096 ≈ 0.8mV — know how to calculate this
- [ ] ADC input impedance — source impedance must be low; add buffer op-amp if needed
- [ ] DAC — digital to analog, use for audio output, analog reference generation
- [ ] **PWM as DAC — RC low-pass filter converts PWM to analog voltage**
- [ ] Op-amp basics — virtual ground, inverting/non-inverting amplifier, comparator mode
- [ ] Noise and decoupling — why ground planes matter, star grounding, avoid ground loops

2.5 Power Supplies & Protection

- [ ] **Linear regulator (LDO) — drops excess voltage as heat; η = Vout/Vin**
- [ ] **Switching regulator (buck/boost) — efficient; more complex; generates EMI**
- [ ] Know: buck steps down (Vout < Vin), boost steps up (Vout > Vin)
- [ ] **Bypass capacitor placement — must be as close to IC power pin as possible**

- [ ] Reverse polarity protection — diode or P-MOSFET in series
- [ ] ESD protection — TVS diodes on external-facing lines (USB, GPIO headers)
- [ ] Inrush current — capacitors cause high current at power-on; use NTC or soft-start
- [ ] Current measurement — shunt resistor + differential amplifier; INA219 style

2.6 PCB & Schematic Reading

- [ ] **Read a schematic — identify power rails, signal flow, component references (R1, C3, U2)**
- [ ] **Understand a datasheet pin table — VCC, GND, GPIO, open-drain, analog, alternate function**
- [ ] **Trace a net — follow a signal from MCU pin through passive components to destination**
- [ ] Decoupling cap placement — why they must be on PCB near the IC, not far away
- [ ] Signal integrity basics — reflection, crosstalk, impedance matching (concept level)
- [ ] Ground plane — why it matters for return current path and EMI

MUST MASTER

Microcontroller Architecture & Bare-Metal Programming

This separates embedded engineers from software engineers. Understand the hardware you are programming.

3.1 ARM Cortex-M Architecture

- [ ] **Cortex-M family: M0 (simple, low cost) / M3 (no FPU) / M4 (DSP + optional FPU) / M7 (dual issue, cache)**
- [ ] **Cortex-A vs Cortex-M — A has MMU, Linux-capable; M is MCU-class, RTOS/bare-metal**
- [ ] **Register set: R0-R12 (general), R13=SP, R14=LR (return address), R15=PC**
- [ ] PSP vs MSP — process stack vs main stack; why RTOS uses PSP per task
- [ ] **Reset sequence: power-on → fetch SP from 0x00000000 → fetch PC from 0x00000004 → execute**
- [ ] Thumb-2 instruction set — 16-bit and 32-bit instructions mixed; why it saves code size
- [ ] Pipeline: fetch-decode-execute; branch penalty; reason behind -O2 vs -O0 code size diff

3.2 Exception & Interrupt System (NVIC)

- [ ] **Exception types: Reset, NMI, HardFault, MemManage, BusFault, UsageFault, SVCall, SysTick**
- [ ] **NVIC — nested vectored interrupt controller; each IRQ has priority, enable bit**
- [ ] **Interrupt priority — lower number = higher priority on ARM Cortex-M**
- [ ] **Preemption — higher priority IRQ can interrupt lower priority ISR**
- [ ] **How interrupt dispatch works: push {R0-R3,R12,LR,PC,xPSR} → load ISR address → execute**
- [ ] **ISR writing rules: no blocking, no malloc, keep execution short**
- [ ] **Volatile for ISR-shared variables — compiler cannot cache in register across ISR boundary**
- [ ] **Atomic access — read-modify-write of a register is NOT atomic; disable IRQ or use LDREX/STREX**
- [ ] **Deferred processing pattern — ISR sets a flag/posts to queue; main loop or task handles work**
- [ ] **SysTick timer — how FreeRTOS uses it for tick, how it generates periodic interrupt**
- [ ] Re-entrant function — safe to call from ISR and main simultaneously; no static/global state

3.3 Peripheral Configuration (Register-Level)

- [ ] **Memory-mapped I/O — peripherals appear as addresses in address space; write to address = write to HW**
- [ ] **GPIO — mode register (input/output/AF/analog), ODR (output), IDR (input), BSRR (atomic set/reset)**
- [ ] **Clock enable — every peripheral clock must be enabled before accessing its registers (RCC on STM32)**
- [ ] **Timer — prescaler, auto-reload register, counter; how to calculate frequency and period**
- [ ] **PWM generation — timer in PWM mode, compare register sets duty cycle**
- [ ] **DMA — what problem it solves (CPU-free data transfer), channel, circular mode, half/full transfer interrupt**
- [ ] ADC — resolution, sampling time, trigger source, DMA readout
- [ ] **Watchdog (IWDG/WWDG) — must kick within window else reset; use case: hung firmware detection**
- [ ] **Clock tree — PLL configuration, AHB/APB bus prescalers; wrong clock = wrong UART baud rate**

3.4 Communication Protocols — Write Timing Diagrams From Memory

- [ ] **UART — async, baud rate, start bit, 8 data bits, optional parity, stop bit; framing error cause**
- [ ] **SPI — 4-wire (MOSI/MISO/SCK/CS); CPOL/CPHA modes 0-3 — draw the 4 timing diagrams**
- [ ] CPOL=0: idle clock LOW. CPOL=1: idle HIGH. CPHA=0: sample on 1st edge, CPHA=1: on 2nd
- [ ] Multi-slave SPI: one CS per slave; why you can't share CS lines
- [ ] **I2C — 2-wire (SDA/SCL); START condition, address byte, ACK/NACK, STOP condition**
- [ ] **7-bit vs 10-bit addressing; calculate pull-up resistor: R = (Vdd-0.4V)/(3mA) for fast mode**
- [ ] **Clock stretching — slave holds SCL low to buy processing time; master must handle it**
- [ ] Multi-master arbitration — both drive SDA, one loses when it sees mismatch — it backs off
- [ ] **I2C repeated START — how to do register read without releasing bus between write and read**
- [ ] CAN bus — differential (CANH/CANL), dominant/recessive bits, arbitration, 11/29-bit IDs
- [ ] CAN error frames, error confinement states: active→passive→bus-off
- [ ] RS-485 — differential, multi-drop up to 32 nodes, half-duplex, direction control pin
- [ ] **JTAG/SWD — JTAG is 4-wire (TDI/TDO/TCK/TMS), SWD is 2-wire (SWDIO/SWCLK); both for debug**

MUST MASTER

RTOS — FreeRTOS Mastery

FreeRTOS is asked at almost every embedded interview. Know it well enough to reason about bugs in code you're shown, not just recite definitions.

4.1 Core Scheduler Concepts

- [ ] **Preemptive scheduling — higher priority task immediately preempts lower; no voluntary yield needed**
- [ ] **Cooperative scheduling — task runs until it yields or blocks; simpler but non-deterministic**
- [ ] **Tick rate — SysTick fires at configTICK_RATE_HZ; each tick scheduler may switch tasks**
- [ ] **Task states: Running / Ready / Blocked (waiting for event/time) / Suspended**
- [ ] **Priority inversion — low priority task holds mutex needed by high priority task; medium priority runs**
- [ ] Classic example: Mars Pathfinder bug — know it, explain it clearly
- [ ] **Priority inheritance — FreeRTOS mutexes (NOT binary semaphores) temporarily boost low-pri task**
- [ ] **Stack per task — statically or dynamically allocated; uxTaskGetStackHighWaterMark() to check usage**
- [ ] Idle task — lowest priority; runs when nothing else ready; use idle hook for power saving
- [ ] configUSE_PREEMPTION, configUSE_TIME_SLICING — what each knob does

4.2 Synchronization Primitives

- [ ] **Binary semaphore — signaling (ISR to task); does NOT have priority inheritance**
- [ ] **Mutex — mutual exclusion; HAS priority inheritance; never take from ISR**
- [ ] **Key difference: binary semaphore for signaling; mutex for protecting shared resource**
- [ ] **Counting semaphore — producer/consumer; counts available items or free slots**
- [ ] **Deadlock — conditions: mutual exclusion, hold and wait, no preemption, circular wait**
- [ ] Deadlock prevention: always acquire locks in same global order
- [ ] **ISR-safe API — xSemaphoreGiveFromISR, xQueueSendFromISR, portYIELD_FROM_ISR**
- [ ] **Always pass and check pxHigherPriorityTaskWoken; call portYIELD if set**
- [ ] Event groups — task can wait on any/all of N bits; xEventGroupWaitBits()
- [ ] Task notifications — lightweight 32-bit signal per task; faster than semaphore
- [ ] **taskENTER_CRITICAL / taskEXIT_CRITICAL — disables interrupts up to configMAX_SYSCALL_INTERRUPT_PRIORITY**

4.3 Queues & Communication

- [ ] **Queue — fixed depth, fixed item size; blocks sender when full, receiver when empty**
- [ ] **Sending structs vs pointers — when to copy by value vs pass pointer (lifetime issues)**
- [ ] xQueueSend vs xQueueSendToFront vs xQueueOverwrite — know when each is appropriate
- [ ] Message buffers — variable-length messages; single reader + single writer only
- [ ] Stream buffers — byte stream (like a UART pipe); single reader + single writer
- [ ] Software timers — timer daemon task runs callbacks; one-shot vs auto-reload
- [ ] Timer callback must be fast — it runs in timer daemon task context, no blocking

4.4 Memory Management

- [ ] heap_1 — allocate only, never free; deterministic, no fragmentation; good for static systems
- [ ] heap_2 — free allowed but no coalescence; fragments over time
- [ ] **heap_4 — free with coalescence; recommended for most projects**
- [ ] heap_5 — heap spans multiple non-contiguous memory regions
- [ ] pvPortMalloc / vPortFree — FreeRTOS wrappers; thread-safe (heap_4/5 use critical section)

4.5 RTOS Debugging Techniques

- [ ] **Stack overflow detection — configCHECK_FOR_STACK_OVERFLOW hook; pattern fill method**
- [ ] **vTaskList() — prints task name, state, priority, stack watermark; invaluable for debugging**
- [ ] Trace — Percepio Tracealyzer or SEGGER SystemView; timeline of task switches and ISRs
- [ ] Assert usage — configASSERT(x) — crash immediately on logic error instead of silently corrupting

Other RTOS to know exist: Zephyr (used in Nordic nRF, Kconfig + DeviceTree based), QNX (automotive, POSIX RT), ThreadX/Azure RTOS (Microsoft, used in Azure Sphere). No depth needed at fresher level.

MUST KNOW WELL

Operating System Fundamentals

Your BTech CSE background helps here. Bridge theory to embedded context — always give embedded examples.

5.1 Process & Thread Management

- [ ] **Process — independent address space, OS-managed; expensive to create/switch**
- [ ] **Thread — shares address space with parent; cheaper context switch**
- [ ] **PCB (Process Control Block) — what the OS stores: registers, PC, SP, open files, memory maps**
- [ ] **Context switch — save current task's registers + PC + SP → restore next task's; cost in cycles**
- [ ] **User space vs kernel space — why the separation exists; privilege levels on ARM (PL0/PL1)**
- [ ] **System call — how user space crosses into kernel: syscall instruction → trap → kernel handler**
- [ ] IPC mechanisms: pipes, message queues, shared memory, sockets — know tradeoffs

5.2 Memory Management

- [ ] **Virtual memory — each process sees its own address space; kernel maps to physical via page tables**
- [ ] **MMU — hardware unit that translates virtual→physical; TLB caches recent translations**
- [ ] **Cortex-A has MMU (Linux), Cortex-M has MPU (RTOS memory protection), M0 has neither**
- [ ] **Page fault — virtual address not in RAM; kernel loads from swap or kills process (segfault)**
- [ ] **MPU on Cortex-M — define regions with access permissions; protect stack from code section overflow**
- [ ] **Cache coherency — DMA writes to RAM; CPU cache may still hold stale data; flush/invalidate cache**
- [ ] Memory-mapped files — file appears in virtual address space; how Linux loads shared libraries

5.3 Scheduling Algorithms

- [ ] **FCFS (First Come First Served) — simple, non-preemptive; convoy effect problem**
- [ ] **SJF (Shortest Job First) — optimal average wait; requires knowing burst time (impossible in practice)**
- [ ] **Round Robin — time quantum; fair; context switch overhead if quantum too small**
- [ ] **Priority scheduling — starvation risk; aging solution (gradually increase priority of waiting tasks)**
- [ ] Multilevel feedback queue — multiple queues, different quanta; new tasks start high, demoted if CPU-bound
- [ ] Real-time scheduling: Rate Monotonic (RMS) — fixed priority based on period; Earliest Deadline First (EDF)

5.4 Concurrency & Synchronization

- [ ] **Race condition — two threads access shared data concurrently, at least one writes; outcome non-deterministic**
- [ ] **Embedded example: ISR writes sensor value while main reads it without protection**
- [ ] **Critical section — region of code that must be atomic; disable interrupts or hold mutex**
- [ ] **Mutex vs spinlock — mutex: sleep while waiting (good for long waits); spinlock: busy-wait (good for short, multicore)**
- [ ] Reader-writer problem — multiple readers OK simultaneously; writer needs exclusive access
- [ ] Dining philosophers — classic deadlock example; solution: order resource acquisition
- [ ] **Semaphore vs mutex — semaphore is a counter, can be released by any thread; mutex has ownership**
- [ ] ABA problem — CAS-based lock-free code; value changed A→B→A; CAS sees A and proceeds incorrectly

MUST KNOW WELL

Embedded Linux & Linux Device Drivers

Mandatory for BSP Engineer, Linux Device Driver, Platform Software Engineer roles. This is your differentiator for higher-paying positions at Qualcomm, NXP, Harman.

6.1 Linux Boot Process

- [ ] **Stage 1 — ROM bootloader (on-chip): initializes DDR, loads secondary bootloader from flash/SD/eMMC**
- [ ] **Stage 2 — U-Boot: initializes more hardware, reads environment variables, loads kernel + DTB into RAM**
- [ ] U-Boot commands: bootz, bootm, setenv, saveenv, tftp, mmc — know basic usage
- [ ] **bootargs in U-Boot — passes kernel command line: root=/dev/mmcblk0p2 rootwait console=ttyS0,115200**
- [ ] **Stage 3 — Kernel decompression and start: uncompress zImage, jump to kernel entry, init arch**
- [ ] **Stage 4 — Kernel init: setup memory, init subsystems, mount rootfs, start PID 1 (init/systemd)**
- [ ] **Device Tree (DTB) — hardware description passed from U-Boot to kernel; replaces board files**
- [ ] **DTS syntax: node name, unit address, compatible string, reg, interrupts, clocks properties**
- [ ] **How to write a device tree node for a simple I2C device (sensor with address and interrupt)**
- [ ] initramfs — minimal RAM-based rootfs loaded before real rootfs; used for pivot-root
- [ ] Yocto Project — build system producing toolchain + rootfs + kernel; layer concept (meta-*)
- [ ] Buildroot — simpler alternative; menuconfig to select packages; produces image directly

6.2 Linux Kernel Architecture

- [ ] **Monolithic kernel with loadable modules — core in single address space; modules extend at runtime**
- [ ] Kernel subsystems: VFS, MM, networking, IPC, scheduler, device model, crypto, security
- [ ] **Kernel module lifecycle: module_init() registered as initcall → insmod calls it → rmmod calls module_exit()**
- [ ] **insmod / rmmod / modprobe / lsmod / modinfo — know each command's purpose**
- [ ] **Kernel memory allocators: kmalloc (physically contiguous, < 128KB), vmalloc (virtually contiguous), kzalloc (zeroed)**
- [ ] **GFP flags: GFP_KERNEL (may sleep), GFP_ATOMIC (ISR-safe, no sleep), GFP_DMA (low memory zone)**
- [ ] **Kernel logging: printk with KERN_INFO, KERN_ERR, KERN_DEBUG; dmesg to read ring buffer**
- [ ] **Kernel concurrency: spinlock (in IRQ context), mutex (sleeps, process context only), RCU (read-copy-update)**
- [ ] **Rule: never sleep while holding a spinlock**
- [ ] **Workqueues — defer work from IRQ context to process context; schedule_work()**
- [ ] Tasklets — softirq-based; run in interrupt context; use for short deferred ISR work

6.3 Interrupt Handling in Linux

- [ ] **Top half — ISR: minimal work, acknowledge HW interrupt, schedule bottom half**
- [ ] **Bottom half options: tasklet (deprecated), workqueue, threaded IRQ (request_threaded_irq)**
- [ ] **request_irq() — register ISR; IRQF_SHARED for shared lines; free_irq() on cleanup**
- [ ] **Threaded IRQ (request_threaded_irq) — top half runs in IRQ context; handler runs as kernel thread**
- [ ] IRQ affinity — bind IRQ to specific CPU core; /proc/irq/N/smp_affinity
- [ ] **Interrupt disable in driver: spin_lock_irqsave / spin_lock_irqrestore — save/restore flags**

6.4 Character Device Driver — Write This From Scratch

- [ ] **file_operations struct — .open, .release, .read, .write, .unlocked_ioctl, .poll**
- [ ] **Register char device: alloc_chrdev_region() → cdev_init() → cdev_add() → class_create() → device_create()**
- [ ] **copy_to_user / copy_from_user — mandatory for user↔kernel data transfer; never dereference user pointer directly**
- [ ] **ioctl — define command codes with _IO/_IOR/_IOW/_IOWR macros; handle in unlocked_ioctl**
- [ ] **wait_queue — block a read() until data available; wake_up() from ISR or write path**
- [ ] **Cleanup in remove/exit: device_destroy → class_destroy → cdev_del → unregister_chrdev_region**
- [ ] udev rules — /etc/udev/rules.d/ — auto-create /dev/mydevice with correct permissions
- [ ] sysfs attributes — expose driver state to userspace via /sys/class/mydev/attribute

6.5 Platform & I2C/SPI Driver Model

- [ ] **Driver model: bus → device → driver; probe() called when device matches driver's compatible string**
- [ ] **Platform driver — for on-SoC peripherals with no auto-discovery; matched via device tree compatible**
- [ ] **platform_driver_register() → probe() gets platform_device with resources (IRQ, IOMEM)**
- [ ] **I2C client driver — i2c_driver with id_table and of_match_table; probe() gets i2c_client**
- [ ] **i2c_master_send / i2c_master_recv / i2c_transfer for register read/write**
- [ ] SPI device driver — spi_driver; spi_transfer and spi_message for transactions
- [ ] **devm_ prefixed APIs — resource-managed; auto-freed on driver remove; use always**
- [ ] Regmap — abstraction for register I/O over I2C/SPI; simplifies driver register access

6.6 BSP & Board Bring-Up Procedure

- [ ] **Step 1: Get UART console working — first sign of life from a new board**
- [ ] **Step 2: Verify U-Boot loads and prints environment correctly**
- [ ] **Step 3: Boot minimal kernel — check dmesg for panic vs successful init**
- [ ] **Step 4: Bring up eMMC/SD card — mount rootfs, check partition table**
- [ ] **Step 5: Bring up Ethernet — ping gateway, confirm PHY link up**
- [ ] **Cross-compile kernel for target: make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig menuconfig zImage**
- [ ] Device tree overlay — extend DTB at runtime; used for capes/hats on BeagleBone/RPi
- [ ] **Pinmux — use device tree pinctrl nodes to set pin function and electrical properties**
- [ ] **Reading a hardware schematic during bring-up — trace MCU → connector → peripheral**

MUST KNOW WELL

Memory Systems, Debugging & Build Tools

7.1 Memory Types & Characteristics

- [ ] **Flash (NOR) — byte addressable, XIP capable, erase by sector (64–256KB blocks); slow erase**
- [ ] **Flash (NAND) — page read/write, block erase; not XIP; needs FTL; used for large storage**
- [ ] **SRAM — fast, no refresh needed; expensive per bit; MCU internal memory**
- [ ] **DRAM/SDRAM — high density, cheap; needs refresh controller; used for Linux systems**
- [ ] EEPROM — byte erasable, limited write cycles (~100k); for config/calibration storage
- [ ] **Cache hierarchy — L1 (per core, ~32KB, ~4 cycle), L2 (shared, ~256KB), L3 (optional)**
- [ ] Cache line — typical 64 bytes; false sharing problem in multicore
- [ ] **Write-back vs write-through cache — difference; when each causes coherency issues with DMA**

7.2 Debugging Tools & Methodology

- [ ] **JTAG/SWD — OpenOCD + GDB; attach to target; set breakpoints, read registers, step through code**
- [ ] **GDB commands: target remote, load, b (breakpoint), c (continue), n (next), s (step), p (print), x (examine memory)**
- [ ] **Logic analyzer — decode SPI/I2C/UART protocol; measure timing; identify glitches**
- [ ] **Oscilloscope — measure voltage levels, timing, signal integrity; probe MCU pin while running**
- [ ] **Multimeter use — continuity check (trace shorts), voltage rail measurement, diode test**
- [ ] **Map file — linker output showing where every function and variable lands in memory; read it for size optimization**
- [ ] Printf debugging pitfalls — UART TX buffer may not flush before crash; add ; use semihosting or RTT for speed
- [ ] Semihosting vs SEGGER RTT — real-time printf over SWD without UART; RTT is much faster
- [ ] ASAN / Valgrind — for host-side testing of embedded code compiled for x86

7.3 Build System & Toolchain

- [ ] **Cross-compilation — building on host (x86) for target (ARM); toolchain = compiler + binutils + sysroot**
- [ ] **Makefile basics — rules, targets, variables, automatic variables ($@, $<, $^)**
- [ ] **CROSS_COMPILE prefix — arm-none-eabi-gcc for bare-metal, aarch64-linux-gnu-gcc for Linux ARM64**
- [ ] **GCC flags for embedded: -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard**
- [ ] Size optimization: -Os; strip debug: -s; separate sections: -ffunction-sections -fdata-sections -Wl,--gc-sections
- [ ] **ELF binary format — sections, symbols; objdump -d (disassemble), nm (symbol table), size (section sizes)**
- [ ] **arm-none-eabi vs arm-linux-gnueabihf — bare-metal (no OS) vs Linux userspace toolchain**
- [ ] CMake basics — CMakeLists.txt, toolchain file for cross-compilation

BE DECENT

DS&A; for Coding Rounds + Networks Basics

MNCs (Qualcomm, TI, NXP, Broadcom) have coding rounds. Focus on embedded-flavored problems and C-based DS. 30–40 targeted problems is enough.

8.1 Data Structures & Algorithms (in C)

- [ ] **Circular buffer — implement from scratch, handle wrap-around, full vs empty distinction**
- [ ] **Linked list: reverse in-place, detect cycle (Floyd's), merge two sorted lists**
- [ ] Stack and queue using arrays — implement all operations with overflow/underflow checks
- [ ] **FSM implementation — state enum + event enum + 2D transition table**
- [ ] Binary search — iterative; know off-by-one issues
- [ ] **Bit manipulation problems: count set bits, find single non-repeating element (XOR trick)**
- [ ] String operations in C: strlen, strcpy, strcat, strcmp — implement from scratch
- [ ] Sorting: know quicksort and mergesort time/space complexity; insertion sort for small arrays
- [ ] Two-pointer technique for array problems

8.2 Computer Networks (for Driver Roles)

- [ ] OSI model 7 layers — name and one-line purpose of each
- [ ] **TCP vs UDP — reliability, ordering, handshake, use cases**
- [ ] TCP three-way handshake: SYN → SYN-ACK → ACK; four-way teardown
- [ ] IP addressing — IPv4, subnet mask, CIDR notation; calculate network address from IP/mask
- [ ] **Socket programming in C: socket() → bind() → listen() → accept() → recv/send() → close()**
- [ ] Ethernet frame: preamble, DST MAC, SRC MAC, EtherType, payload, FCS
- [ ] PHY vs MAC — physical layer chip vs MAC controller in SoC; MDIO bus for PHY control
- [ ] Linux network driver: net_device struct, ndo_start_xmit, NAPI for RX polling

AWARENESS

Hardware Security & Cryptography Awareness

You don't need depth at fresher interviews, but mentioning these at NXP, Qualcomm, or TI signals your long-term direction and gets noticed.

9.1 Security Concepts

- [ ] Secure boot — chain of trust from ROM bootloader; each stage verifies signature of next stage
- [ ] Root of trust — immutable key stored in OTP/eFuse; foundation of entire trust chain
- [ ] TrustZone (Cortex-A) — secure world vs normal world; TEE (Trusted Execution Environment)
- [ ] HSM / TPM / Secure Element — dedicated hardware for key storage and crypto operations
- [ ] OP-TEE — open source TEE OS running in TrustZone secure world
- [ ] Side-channel attacks — timing attack, power analysis; hardware countermeasures
- [ ] NXP EdgeLock SE050 — secure element IC for IoT; what it provides (key storage, crypto accelerator)

9.2 Cryptography Basics

- [ ] Symmetric encryption: AES-128/256 — same key encrypt/decrypt; fast; key distribution problem
- [ ] Asymmetric: RSA, ECC — public/private key pair; ECC preferred for embedded (smaller keys)
- [ ] Hash functions: SHA-256 — one-way, fixed output, collision resistant; used for integrity
- [ ] HMAC — keyed hash for message authentication; not just integrity but also authentication
- [ ] RNG quality — hardware TRNG (true random) vs PRNG (deterministic); why hardware source matters in embedded
- [ ] Crypto accelerators in MCUs — STM32 AES hardware engine, NXP CAAM; why HW acceleration matters
- [ ] Zero-knowledge proofs — concept: prove you know X without revealing X; relevant to your Web3 interest

MUST MASTER

2-Month Study Plan

Week	Focus	Goal
Week 1	C Programming — Sections 1.1 to 1.4	Write pointer + memory questions cold. Implement ring buffer + linked list.
Week 2	C Programming — Sections 1.5, 1.6 + Basic Electronics Sec 2 topics 2.1–2.3	Finish all C topics. Understand pull-ups, voltage levels, register reading.
Week 3	MCU Architecture — All of Section 3	Explain reset flow, interrupt system, SPI/I2C timing from memory. Write ISR code.
Week 4	FreeRTOS — All of Section 4	Code a FreeRTOS project with queues, mutexes, ISR↔task handoff on real hardware.
Week 5	Embedded Linux — Sections 6.1 + 6.2 + 6.3	Know boot process cold. Write and load a basic kernel module on Raspberry Pi.
Week 6	Linux Device Drivers — Sections 6.4 + 6.5 + 6.6	Write a working character driver. Understand platform driver probe/remove lifecycle.
Week 7	OS Fundamentals (Sec 5) + Networks (Sec 8.2) + Debug (Sec 8.4)	Complete Sec 8.3 + 8.4. LeetCode / GFG problems. All in C. Focus on bit ops and linked lists.
Week 8	Review + Memory & Debug (Sec 7) + Electronics Sec 2.4–2.6	Mock interviews. Fix weak spots. Read one datasheet end-to-end. GDB practice session

Rules for this period

- *Never tick a box because you read it. Tick it when you can explain it out loud or write the code without looking.*
- *Do not skip hardware: buy or borrow an STM32 Nucleo (cheap) and a Raspberry Pi. Theory alone is not enough.*
- *For every protocol (SPI, I2C, UART) — read the timing diagram in a real datasheet, not just a tutorial.*
- *Target Qualcomm, TI, NXP, Harman, Sasken, VVDN for high packages. Their interview bar is high — prepare accordingly.*
- *Before interviews: do 2–3 mock sessions. Record yourself. Can you explain priority inversion clearly in 90 seconds?*
- *Long-term: your hardware security + Web3 (DePIN) path is coherent. Keep one side project alive throughout — a crypto-enabled hardware node ties it all together for German Masters applications.*

Embedded Systems Study Checklist · Generated for CDAC ACTS Pune → Industry → Masters (Germany)